

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: df = pd.read_csv(r"Data.csv")
df
```

```
Out[2]:
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No
1	Spain	27.0	48000.0	Yes
2	Germany	30.0	54000.0	No
3	Spain	38.0	61000.0	No
4	Germany	40.0	NaN	Yes
5	France	35.0	58000.0	Yes
6	Spain	NaN	52000.0	No
7	France	48.0	79000.0	Yes
8	Germany	50.0	83000.0	No
9	France	37.0	67000.0	Yes

```
In [3]: #Numpy
X_raw=df.iloc[:, :-1].values
Y_raw=df.iloc[:, -1].values
print(X_raw)
print(Y_raw)
```

```
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 nan]
 ['France' 35.0 58000.0]
 ['Spain' nan 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
['No' 'Yes' 'No' 'No' 'Yes' 'Yes' 'No' 'Yes' 'No' 'Yes']
```

```
In [4]: #Pandas
dfX_raw=df.iloc[:, :-1]
dfY_raw=df.iloc[:, -1]

print(dfX_raw)
dfY_raw
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
Out[4]: 0    No
1    Yes
2    No
3    No
4    Yes
5    Yes
6    No
7    Yes
8    No
9    Yes
Name: Purchased, dtype: object
```

```
In [5]: #Copy the data to another variable
dfX = dfX_raw.copy()
seriesY = dfY_raw.copy()
X = X_raw.copy()
Y = Y_raw.copy()
```

```
In [6]: print(dfX.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Country     10 non-null     object
1   Age         9 non-null      float64
2   Salary      9 non-null      float64
dtypes: float64(2), object(1)
memory usage: 368.0+ bytes
None
```

In [7]: `print(X.info())`

```
-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_11104\3543332853.py in <module>
----> 1 print(X.info())

AttributeError: 'numpy.ndarray' object has no attribute 'info'
```

In [8]: `#Change numpy array to DataFrame`
`X = pd.DataFrame(X)`
`X.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0    0         10 non-null    object
 1    1         9 non-null     object
 2    2         9 non-null     object
dtypes: object(3)
memory usage: 368.0+ bytes
```

In [9]: `#Then Change to Numpy`
`X = X.values`
`X`

Out[9]: `array([[ 'France', 44.0, 72000.0],`
 `['Spain', 27.0, 48000.0],`
 `['Germany', 30.0, 54000.0],`
 `['Spain', 38.0, 61000.0],`
 `['Germany', 40.0, nan],`
 `['France', 35.0, 58000.0],`
 `['Spain', nan, 52000.0],`
 `['France', 48.0, 79000.0],`
 `['Germany', 50.0, 83000.0],`
 `['France', 37.0, 67000.0]], dtype=object)`

In [10]: `X.info()`

```
-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_11104\2718079001.py in <module>
----> 1 X.info()

AttributeError: 'numpy.ndarray' object has no attribute 'info'
```

```
In [11]: #Counting the number of null values in each attribute
print(dfX.isnull().sum())
```

```
Country    0
Age        1
Salary     1
dtype: int64
```

```
In [12]: print(X.isnull().sum()) #Error in Numpy Array
```

```
-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_11104\3706335114.py in <module>
----> 1 print(X.isnull().sum()) #Error in Numpy Array

AttributeError: 'numpy.ndarray' object has no attribute 'isnull'
```

```
In [13]: dfX_raw
```

```
Out[13]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

In [14]: dfX

Out[14]:

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

In [15]: dfX = dfX.fillna(0) *#Replace the NAN with 0*
dfX

Out[15]:

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	0.0
5	France	35.0	58000.0
6	Spain	0.0	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
In [16]: dfX = dfX.replace(0, np.nan) # Replace the 0 with NAN
dfX
```

```
Out[16]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
In [17]: #index of a particular Attribute

index_no = dfX.columns.get_loc('Age')
index_no
```

```
Out[17]: 1
```

```
In [18]: # Filling the missing places with a constant value

dfX.iloc[:,1] = dfX.iloc[:,1].fillna(5)
dfX
```

```
Out[18]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	5.0	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

In [19]: *#Getting the Old Data*

```
dfX = dfX_raw.copy()  
dfX
```

Out[19]:

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```

In [20]: # Replace with Mean Value

# SimpleImputer is the class which is used to handle the missing values with simple
#dataframe
dfX = dfX_raw.copy()

from sklearn. impute import SimpleImputer

imputer = SimpleImputer(missing_values=np.nan, strategy='mean')

#imputer.fit(dfX. iloc[:, 1:3]) #Calculates
#dfX. iloc[:, 1:3] = imputer. transform(dfX. iloc[:, 1:3]) # Assign the calculated
#dfX. iloc[:, 1:3] = imputer.fit(dfX. iloc[ :, 1:3]).transform(dfX. iloc[:, 1:3])

dfX.iloc[:, 1:3] = imputer.fit_transform(dfX.iloc[:, 1:3])
dfX

```

Out[20]:

	Country	Age	Salary
0	France	44.000000	72000.000000
1	Spain	27.000000	48000.000000
2	Germany	30.000000	54000.000000
3	Spain	38.000000	61000.000000
4	Germany	40.000000	63777.777778
5	France	35.000000	58000.000000
6	Spain	38.777778	52000.000000
7	France	48.000000	79000.000000
8	Germany	50.000000	83000.000000
9	France	37.000000	67000.000000

In [21]: *#Getting the Old Data*

```
dfX = dfX_raw.copy()  
dfX
```

Out[21]:

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	NaN
5	France	35.0	58000.0
6	Spain	NaN	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
In [22]: dfX = dfX_raw.copy()
from sklearn. impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy='most_frequent')

#imputer.fit(dfX. iloc[:, 1:3])
#dfX. iloc[:, 1:3] = imputer.transform(dfX. iloc[:, 1:3])

#dfX. iloc[:, 1:3] = imputer.fit(dfX. iloc[ :, 1:3]). transform(dfX. iloc[:, 1:3])

dfX.iloc[:, 1:3] = imputer.fit_transform(dfX.iloc[:, 1:3])
dfX
```

C:\Users\SILICON\anaconda3\lib\site-packages\sklearn\impute_base.py:49: Future Warning: Unlike other reduction functions (e.g. `skew`, `kurtosis`), the default behavior of `mode` typically preserves the axis it acts along. In SciPy 1.11.0, this behavior will change: the default value of `keepdims` will become False, the `axis` over which the statistic is taken will be eliminated, and the value None will no longer be accepted. Set `keepdims` to True or False to avoid this warning.

```
mode = stats.mode(array)
```

Out[22]:

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	48000.0
5	France	35.0	58000.0
6	Spain	27.0	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
In [23]: #dataframe
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy='constant', fill_value=5)
imputer.fit(dfX.iloc[:, 1:3])
dfX.iloc[:, 1:3] = imputer.fit(dfX.iloc[:, 1:3]).transform(dfX.iloc[:, 1:3])
dfX
```

```
Out[23]:
```

	Country	Age	Salary
0	France	44.0	72000.0
1	Spain	27.0	48000.0
2	Germany	30.0	54000.0
3	Spain	38.0	61000.0
4	Germany	40.0	48000.0
5	France	35.0	58000.0
6	Spain	27.0	52000.0
7	France	48.0	79000.0
8	Germany	50.0	83000.0
9	France	37.0	67000.0

```
In [24]: X=X_raw.copy()
from sklearn.impute import SimpleImputer
imputer=SimpleImputer(missing_values=np.nan,strategy='mean')
X[:,1:3]=imputer.fit_transform(X[:,1:3])
print(X)
```

```
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 63777.77777777778]
 ['France' 35.0 58000.0]
 ['Spain' 38.77777777777778 52000.0]
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]
```

Encoding Categorical Data

```
In [25]: #pandas
dfX=pd.get_dummies(dfX,columns=['Country'])#one hot encoding(For Each Category)
dfX
```

```
Out[25]:
```

	Age	Salary	Country_France	Country_Germany	Country_Spain
0	44.0	72000.0	1	0	0
1	27.0	48000.0	0	0	1
2	30.0	54000.0	0	1	0
3	38.0	61000.0	0	0	1
4	40.0	48000.0	0	1	0
5	35.0	58000.0	1	0	0
6	27.0	52000.0	0	0	1
7	48.0	79000.0	1	0	0
8	50.0	83000.0	0	1	0
9	37.0	67000.0	1	0	0

```
In [26]: #numpy
X = pd.get_dummies(X, columns=[0])      # Will not work on numpy array
X
```

```
-----
ValueError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_11104\3378575693.py in <module>
      1 #numpy
----> 2 X = pd.get_dummies(X, columns=[0])      # Will not work on numpy array
      3 X

~\anaconda3\lib\site-packages\pandas\core\reshape\reshape.py in get_dummies(data, prefix, prefix_sep, dummy_na, columns, sparse, drop_first, dtype)
    987         result = concat([with_dummies], axis=1)
    988     else:
--> 989         result = _get_dummies_1d(
    990             data,
    991             prefix,

~\anaconda3\lib\site-packages\pandas\core\reshape\reshape.py in _get_dummies_1d(data, prefix, prefix_sep, dummy_na, sparse, drop_first, dtype)
   1011
   1012     # Series avoids inconsistent NaN handling
-> 1013     codes, levels = factorize_from_iterable(Series(data))
   1014
   1015     if dtype is None:

~\anaconda3\lib\site-packages\pandas\core\series.py in __init__(self, data, index, dtype, name, copy, fastpath)
    449         data = data.copy()
    450     else:
--> 451         data = sanitize_array(data, index, dtype, copy)
    452
    453         manager = get_option("mode.data_manager")

~\anaconda3\lib\site-packages\pandas\core\construction.py in sanitize_array(data, index, dtype, copy, raise_cast_failure, allow_2d)
    599         subarr = maybe_infer_to_datetimelike(subarr)
    600
--> 601         subarr = _sanitize_ndim(subarr, data, dtype, index, allow_2d=allow_
    602         2d)
    603         if isinstance(subarr, np.ndarray):

~\anaconda3\lib\site-packages\pandas\core\construction.py in _sanitize_ndim(result, data, dtype, index, allow_2d)
    650         if allow_2d:
    651             return result
--> 652         raise ValueError("Data must be 1-dimensional")
    653         if is_object_dtype(dtype) and isinstance(dtype, ExtensionDtype):
    654             # i.e. PandasDtype("O")

ValueError: Data must be 1-dimensional
```

```
In [27]: # get the raw data
#Copy the data to another variable
dfX = dfX_raw.copy()
seriesY = dfY_raw.copy()
X = X_raw.copy()
Y = Y_raw.copy()
```

```
In [30]: from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
#ct=ColumnTransformer(transformers=[('encoder', 'drop', [0])], remainder="passthrough")
ct=ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])], remainder="passthrough")
#ct=ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])])
X=np.array(ct.fit_transform(X))
X
```

```
Out[30]: array([[0.0, 1.0, 0.0, 0.0, 44.0, 72000.0],
 [1.0, 0.0, 0.0, 1.0, 27.0, 48000.0],
 [1.0, 0.0, 1.0, 0.0, 30.0, 54000.0],
 [1.0, 0.0, 0.0, 1.0, 38.0, 61000.0],
 [1.0, 0.0, 1.0, 0.0, 40.0, nan],
 [0.0, 1.0, 0.0, 0.0, 35.0, 58000.0],
 [1.0, 0.0, 0.0, 1.0, nan, 52000.0],
 [0.0, 1.0, 0.0, 0.0, 48.0, 79000.0],
 [1.0, 0.0, 1.0, 0.0, 50.0, 83000.0],
 [0.0, 1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

```
In [32]: from sklearn.compose import ColumnTransformer
# from sklearn.preprocessing import OneHotEncoder #This is Commented Out
#ct=ColumnTransformer(transformers=[('encoder', 'drop', [0])], remainder="passthrough")
ct=ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])], remainder="passthrough")
#ct=ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])])
X=np.array(ct.fit_transform(X))
X

#The 1.0 is changed to 0.0 and vice versa
```

```
Out[32]: array([[0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 44.0, 72000.0],
 [1.0, 0.0, 1.0, 0.0, 0.0, 1.0, 27.0, 48000.0],
 [1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 30.0, 54000.0],
 [1.0, 0.0, 1.0, 0.0, 0.0, 1.0, 38.0, 61000.0],
 [1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 40.0, nan],
 [0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 35.0, 58000.0],
 [1.0, 0.0, 1.0, 0.0, 0.0, 1.0, nan, 52000.0],
 [0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 48.0, 79000.0],
 [1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 50.0, 83000.0],
 [0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

Encoding the Dependent Variable

```
In [35]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
Y = le.fit_transform(Y)
print(Y)
```

```
[0 1 0 0 1 1 0 1 0 1]
```

```
In [41]: #Encoding will be done according to alphabetical order.
#Label encoding will use numbers from 0 to n-1.
le.fit(["india","paris", "paris", "tokyo", "amsterdam"]) #encoding is done
print(le.transform(["india", "amsterdam", "tokyo", "paris"])) #transformation is
```

```
[1 0 3 2]
```

```
In [37]: print(le.fit_transform(["india", "paris"]))
```

```
[0 1]
```

```
In [38]: print(le.transform(["paris", "india"]))
```

```
[1 0]
```

```
In [42]: print(le.transform(["tokyo", "amsterdam"]))
```

```
[3 0]
```

Splitting the DataSet into Train, Test Split

```
In [43]: from sklearn.model_selection import train_test_split
X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.2,random_state=1)
```

```
In [44]: print(X_train)
```

```
[[1.0 0.0 1.0 0.0 0.0 1.0 nan 52000.0]
 [1.0 0.0 1.0 0.0 1.0 0.0 40.0 nan]
 [0.0 1.0 0.0 1.0 0.0 0.0 44.0 72000.0]
 [1.0 0.0 1.0 0.0 0.0 1.0 38.0 61000.0]
 [1.0 0.0 1.0 0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 1.0 0.0 0.0 48.0 79000.0]
 [1.0 0.0 1.0 0.0 1.0 0.0 50.0 83000.0]
 [0.0 1.0 0.0 1.0 0.0 0.0 35.0 58000.0]]
```

```
In [45]: print(X_test)
```

```
[[1.0 0.0 1.0 0.0 1.0 0.0 30.0 54000.0]
 [0.0 1.0 0.0 1.0 0.0 0.0 37.0 67000.0]]
```

In [46]: `print(Y_train)`

```
[0 1 0 0 1 1 0 1]
```

In [47]: `print(Y_test)`

```
[0 1]
```

```
In [ ]: # The Standard Scaler is Applied on the Features having Numerical Value.
# The Formula  $[Z = (x - U(\mu)) / S(\sigma)]$ 
#           X = Original Value
#           U = Mean Value
#           S = Standard Deviation
#           Z = Z-Score Value
```

Feature Scaling

In [48]: `from sklearn.preprocessing import StandardScaler`
`sc=StandardScaler()`
`X_train[:,3:]=sc.fit_transform(X_train[:,3:])`
`X_test[:,3:]=sc.transform(X_test[:,3:])`

In [49]: `print(X_train)`

```
[[1.0 0.0 1.0 -0.7745966692414834 -0.5773502691896258 1.2909944487358056
  nan -1.0182239953527132]
 [1.0 0.0 1.0 -0.7745966692414834 1.7320508075688774 -0.7745966692414834
 -0.038910211282047996 nan]
 [0.0 1.0 0.0 1.2909944487358056 -0.5773502691896258 -0.7745966692414834
 0.5058327466666259 0.583476671494251]
 [1.0 0.0 1.0 -0.7745966692414834 -0.5773502691896258 1.2909944487358056
 -0.3112816902563849 -0.2974586952715793]
 [1.0 0.0 1.0 -0.7745966692414834 -0.5773502691896258 1.2909944487358056
 -1.809324824615238 -1.3385641287221062]
 [0.0 1.0 0.0 1.2909944487358056 -0.5773502691896258 -0.7745966692414834
 1.0505757046152997 1.1440719048906884]
 [1.0 0.0 1.0 -0.7745966692414834 1.7320508075688774 -0.7745966692414834
 1.3229471835896367 1.4644120382600814]
 [0.0 1.0 0.0 1.2909944487358056 -0.5773502691896258 -0.7745966692414834
 -0.7198389087178904 -0.537713795298624]]
```

In [50]: `print(X_test)`

```
[[1.0 0.0 1.0 -0.7745966692414834 1.7320508075688774 -0.7745966692414834
 -1.4007676061537326 -0.8580539286680168]
 [0.0 1.0 0.0 1.2909944487358056 -0.5773502691896258 -0.7745966692414834
 -0.4474674297435534 0.18305150478250995]]
```

In []:

